CSE 396 – Spring 2025
Module Requirements-Documentation



SD Belt

(Scan & Detect Belt)

Group 15 a.k.a. “Belt Innovations”

18.04.2025

Belt Innovations	Image Processing & AI	Hardware	Mobile Application	Desktop Application	Database and Backend
Ahmet Abdulgaffar Fettahoğlu			+		+
Büşra Sarı	+				+
Cemil Fatih Yol	+		+		
Kağan Çakıroğlu		+		+	
Mehmet Hayrullah Özkul			+		+
Mert Emir Şeker		+		+	
Osmancan Bozali			+		+
Serkan Efe Çamoğlu	+			+	
Ziya Kadir Tokluoğlu	+	+			

Table of Contents

Modules 5

1. **YOLO-Based Image Processing Module** 5
   - 1.1. **Introduction** 5
   - 1.2. **Technology Approach** 5
   - 1.3. **Dataset and Data Handling** 6
   - 1.4. **Preprocessing & Postprocessing** 6
   - 1.5. **Model Training and Fine-Tuning** 7
   - 1.6. **Deployment on Raspberry Pi** 7
   - 1.7. **Future Improvements** 7
   - 1.8. **Hardware-Camera Communication System** 8
   - 1.9. **API Integration for Data Transmission** 10
   - 1.10. **Test Cases for YOLO-Based Image Processing Module** 12
2. **Desktop Application Module** 14
   - 2.1. **Overview** 14
   - 2.2. **Implemented Features** 14
   - 2.3. **Possible Future Features** 15
   - 2.4. **Technologies Used** 15
   - 2.5. **Message Format Examples** 15
   - 2.6. **Desktop Figma Designs** 16
   - 2.7. **Test Cases for Desktop Application Module** 18
3. **Hardware Module** 20
   - 3.1. **Overview** 20
   - 3.2. **SD Belt Videos** 20
   - 3.3. **SD Belt Workflow** 22
   - 3.4. **Test Cases for Hardware Module** 23
4. **Mobile Application Module** 25
   - 4.1. **Introduction** 25
   - 4.2. **Technology Approach** 25
   - 4.3. **Current Progress** 26
   - 4.4. **Development Roadmap** 28
   - 4.5. **Integration with Other Modules** 29
   - 4.6. **Test Cases for Mobile Application Module** 29
5. **Database and Backend Module** 30
   - 5.1. **Introduction** 30
   - 5.2. **Technology Approach** 30
   - 5.3. **Backend Architecture** 30

5.4. Development Roadmap..... 31

5.5. Integration with Other Modules The backend coordinates interactions across all SD Belt modules: 32

5.6. Test Cases for Database and Backend Module..... 32

Modules

1. YOLO-Based Image Processing Module

1.1. Introduction

This project aims to develop an automated image processing system for quality control in a manufacturing environment using YOLO (You Only Look Once) and Raspberry Pi. The goal is to accurately identify and quantify defects in products, focusing specifically on fruits, which are ideal candidates due to the availability of both clean and damaged examples. The system will detect and highlight defects in real time, providing actionable results directly on the production line.

1.2. Technology Approach

1.2.1. Technology Stack

- **Hardware**: Raspberry Pi 4, Standard USB webcams (cost-efficient alternative to industrial cameras)
- **AI Framework**: YOLOv8n (Ultralytics Nano version, optimized for edge devices like Raspberry Pi)
- **Language**: C++ for model execution and system control
- **Image Processing Library**: OpenCV (for preprocessing, masking, and visualization)

1.2.2. Layered Detection Architecture

The detection system will follow a two-stage pipeline:

- **Fruit Detection Layer**: Identify all fruits in the frame.
- **Defect Detection Layer**: Detect and localize defects within each identified fruit using bounding boxes.

This layered architecture enables multi-fruit handling and refined defect localization.

1.3. Dataset and Data Handling

1.3.1. Dataset Collection

We will use a combination of publicly available datasets and a custom-generated dataset. Public datasets will provide a base level of variety, while our custom dataset will focus on specific fruit types and defect scenarios relevant to our setup:

- Capture images of multiple fruit types under various conditions.
- Adjust lighting and background to assist model learning.
- Simulate and label real-world defect conditions.

1.3.2. Data Augmentation

To improve generalization and robustness:

- Rotation and flipping
- Brightness and contrast adjustments
- Gaussian noise and blur
- Occlusion simulation (partial covering)

1.4. Preprocessing & Postprocessing

1.4.1. Preprocessing

- **Masking**: Apply masks to isolate fruits from the background using color thresholding and contour analysis.
- **Resizing**: Resize images to YOLOv8n's expected input size (e.g., 640x640).
- **Normalization**: Normalize pixel values according to model input requirements.

1.4.2. Postprocessing

- **Bounding Box Filtering**: Filter predictions based on confidence threshold.
- **Defect Highlighting**: Overlay bounding boxes around detected defect regions.
- **Defect Area Calculation**: Estimate the defect area as a percentage of each fruit’s bounding box for reporting.

1.5. Model Training and Fine-Tuning

1.5.1. Training Strategy

- Use YOLOv8n pretrained weights.
- Fine-tune the model using a combination of public and custom datasets.
- Incorporate diverse scenarios to improve multi-fruit detection and defect localization.

1.5.2. Hyperparameter Tuning

- Optimize learning rate, batch size, and training epochs.
- Refine IOU thresholds and anchor boxes (if needed) to boost accuracy.

1.5.3. Evaluation Metrics

- Precision, Recall, and mAP (mean Average Precision)
- Real-time performance benchmarks on Raspberry Pi

1.6. Deployment on Raspberry Pi

1.6.1. Optimization

- Use YOLOv8n for efficient edge deployment.
- Leverage OpenCV-DNN or Ultralytics runtime optimized for Raspberry Pi.

1.6.2. Real-Time Pipeline

- Capture frames using 2–4 connected cameras
- Preprocess frames using OpenCV
- Run YOLO inference
- Postprocess and visualize results or trigger actuators

1.7. Future Improvements

- Expand dataset with more fruit types and defect conditions
- Integrate centralized cloud storage and update system
- Add support for multiple processing nodes and camera stitching

This project sets the foundation for a scalable, cost-effective visual quality control solution using embedded AI.

1.8. Hardware-Camera Communication System

1.8.1. Key Responsibilities:

Multi-Camera Setup and Configuration:

- Establish and maintain connections with two webcams positioned strategically around the conveyor belt.
- Configure camera parameters such as resolution and frame rate to optimize image quality for defect detection.

Thread-Based Parallel Image Acquisition:

- Implement a multi-threaded architecture that dedicates independent threads to each camera.
- Ensure cameras can operate simultaneously to capture different angles of products on the conveyor belt.
- Address current low frame rate issues to improve overall system performance.

Image Acquisition Pipeline:

- Develop efficient frame capture and transfer mechanisms compatible with the YOLOv8n model requirements.
- Prepare captured images for processing by the defect detection system.

1.8.2. Implementation Details:

Camera Selection Rationale: Standard webcams are chosen over industrial cameras due to their cost efficiency while still providing adequate image quality for defect detection.

Camera Deployment Strategy:

- **Initial Configuration**: Two webcams are deployed at a 180-degree angle from each other, positioned on directly opposite sides of the conveyor belt. This arrangement ensures complete side coverage of products as they move along the belt.
- **Third Camera Option**: If necessary, a third camera will be mounted directly above the conveyor belt to provide a top-down view, creating a comprehensive three-dimensional capture system.

1.8.3. Performance Optimization Strategy:

a. **Current Status**: Currently, we are experiencing low frames-per-second (FPS) from the Raspberry Pi cameras, which is a limitation we're actively addressing.

b. **Thread-Based Solution**: Our multi-threaded architecture separates camera operations from processing, allowing cameras to capture frames continuously while defect detection occurs in parallel.

1.8.4. Thread Architecture Strategy:

The system implements a multi-threaded approach for camera management with these key features:

- **Dedicated Camera Threads**: Each webcam operates in its own dedicated thread, allowing simultaneous frame capture from different angles around the conveyor belt.
- **180-Degree Camera Positioning**: Two primary cameras are positioned at 180 degrees from each other on opposite sides of the conveyor belt, providing complete side views of products.
- **Optional Top Camera**: If necessary, a third camera positioned above the conveyor belt provides a top-down view, completing the visual coverage of products.
- **Centralized Processing**: A dedicated processing thread handles the YOLOv8n model execution on the Raspberry Pi AI Kit, keeping intensive processing separate from frame capture.
- **Continuous Operation**: The thread architecture ensures that frame capture continues uninterrupted even during intensive processing operations.

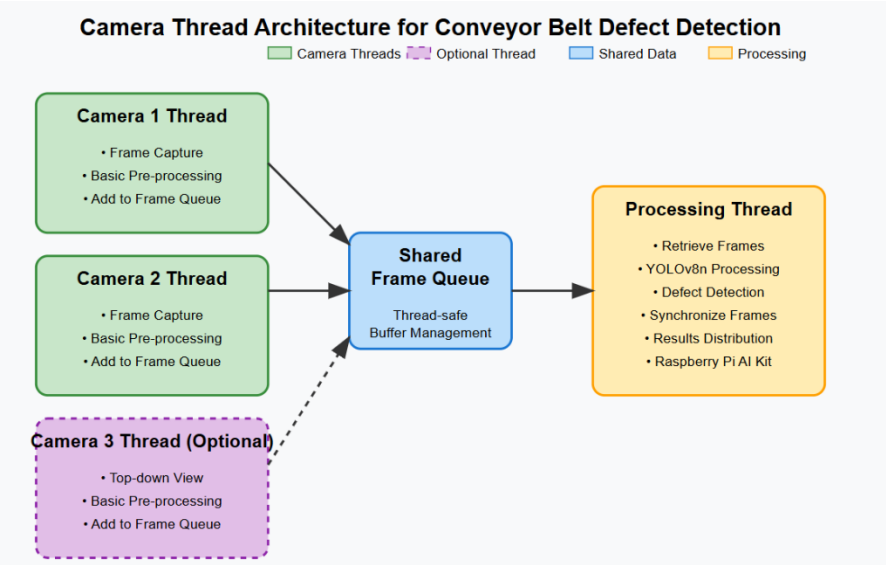


Figure 1: Thread Architecture for Multi-Camera Defect Detection System

1.8.5. Real-time Acquisition Approach:

- **Current Status**: Currently, we are experiencing low frames-per-second (FPS) from the Raspberry Pi cameras, which we aim to improve through our thread architecture.
- **Camera Positioning**: Two cameras positioned on opposite sides of the conveyor belt, with an optional third camera above for complete product coverage.
- **Target Frame Rate**: Working toward achieving higher frame rates by optimizing the thread structure and utilizing the Raspberry Pi AI Kit.

After processing images using the YOLO model in C++ to detect defective products on the production line, the resulting data must be transmitted to various parts of the system such as the desktop application, mobile app, and backend database.

1.9. API Integration for Data Transmission

1.9.1. Data Formatting and Communication

Data Generation:

The image processing unit produces detection results including:

- Product ID
- Detection timestamp
- Defect presence
- Confidence scores
- Bounding box coordinates for the detected defects

Data Serialization:

The detection results are organized into a JSON object.

- JSON is chosen for its language-agnostic properties and ease of integration, making it an ideal format to bridge communication among diverse system modules.

1.9.2. Using the API for Data Transmission

API Role:

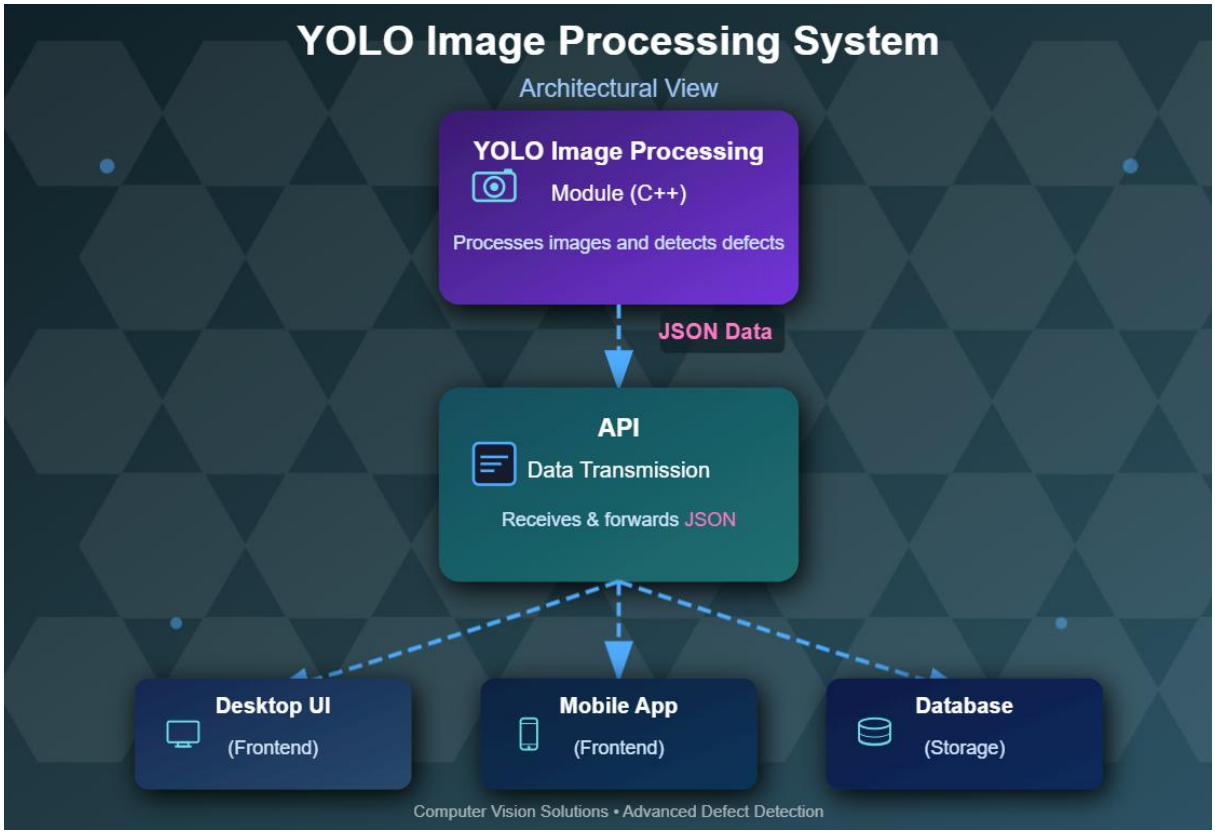
A dedicated API (primarily in C++ with supportive tools) is responsible for handling data transmission. It acts as a central dispatcher, ensuring that all detection data is consistently forwarded to the designated endpoints.

Process Workflow:

1. **Data Collection:**
   The image processing module captures real-time detection results.
2. **Data Serialization:**
   These results are organized into a JSON object.
3. **Data Transmission:**
   The JSON object is transmitted via an HTTP POST request using the API.
4. **Endpoint Consumption:**
   The receiving modules, desktop app, mobile app, and backend systems, parse the JSON data to update dashboards, trigger notifications, or log the data for further analysis.

1.9.3. Architectural Diagram

Below is a conceptual illustration of how the YOLO Image Processing module interacts with the API to distribute detection results to the desktop UI, mobile app, and database.



1.9.4. Cross-Platform and Performance Considerations

- **High Performance:**
  The implementation ensures efficient real-time data processing crucial for production-line quality control.
- **Cross-Platform Compatibility:**
  When properly configured, the API can run seamlessly on both Linux and Windows platforms, allowing broad deployment options.
- **Direct Integration:**
  Maintaining the data flow within a primarily C++ environment simplifies integration and reduces the need to interface multiple languages.

1.10. Test Cases for YOLO-Based Image Processing Module

Feature	Purpose	Test Input / Action	Expected Result	Result
Fruit Detection	Verify that all fruits in a frame are correctly detected	A single image containing 3 apples and 2 oranges on a plain background	Exactly 5 bounding boxes are returned (3 for apples, 2 for oranges), each with confidence ≥ 0.5	Planned
Defect Identification	Ensure that known defects are flagged	An image of an apple with a brown bruise region, plus one unblemished apple	One defect bounding box around the bruise with class "defect" and confidence ≥ 0.6; the other apple has no defect box	Planned
No-False-Positive Detection	Confirm module does not falsely flag pristine items	An image of 5 flawless fruits	Zero "defect" boxes; exactly 5 "fruit" boxes	Planned
Multi-Camera Stream Handling	Validate that two camera feeds can be ingested and processed in parallel without dropping frames	Two simultaneous video streams (cam1.mp4 and cam2.mp4) each containing 10 frames of moving fruits	All 20 frames (10 per stream) processed; inference time per frame ≤ 200 ms; no dropped frames	Planned
Confidence Threshold Filtering	Test that detections below configured threshold are discarded	An image with one fruit detection at confidence=0.45 (below threshold T)	The low-confidence detection is filtered out; no bounding box returned	Planned
JSON Output & Schema Validation	Check that module outputs detection results in correct JSON format	Run detection on a test image and capture the module's JSON output	JSON contains keys: `product_id`, `timestamp`, `defect_present`, `confidence`, `bbox` (with 4 coords); types and nesting match the API spec	Planned
Real-Time Throughput	Verify pipeline meets minimum FPS requirement	Feed a 60-second 30 FPS test video to the	Sustained processing at ≥ 15 FPS over the entire video	Planned

		module on Raspberry Pi		
Low-Light / High-Glare Robustness	Ensure detection still works under challenging lighting	A pair of test images of fruits under dim light (~50 lux) and under direct glare	Fruits still detected with confidence ≥ 0.5; defects (if present) detected with confidence ≥ 0.6	Planned
Error Handling: Missing Frame	Confirm module gracefully handles a dropped or empty frame	Inject a blank frame (all zeros) in the middle of a camera stream	Module logs a warning “Empty frame received” and skips inference without crashing; subsequent frames continue to process normally	Planned
Defect Area Calculation	Validate that post-processing correctly computes defect area percentage	An image where defect covers exactly 25% of the fruit bounding box	JSON field `defect_area_pct` reported as ~25.0 ± 1%; no other fields affected	Planned

2. Desktop Application Module

2.1. Overview

The Desktop Application Module is a core part of the overall system that allows operators to interact with the hardware in real time. It consists of two main subcomponents:

1. Graphical User Interface (GUI): The visual layer that displays system information, logs, notifications, and control buttons to the operator.

2. Communication Layer (WebSocket Submodule): The underlying mechanism that enables real-time, bidirectional communication between the desktop system and the Raspberry Pi.

This section focuses on the Communication Layer, which has been developed in C++ using the QtWebSockets module to maintain an efficient, low-latency WebSocket connection to the Raspberry Pi.

2.2. Implemented Features

Notify:

The module receives real-time system notifications (e.g., alerts, status changes, error messages) from the Raspberry Pi and forwards them to the GUI for immediate display. These notifications may also be forwarded to the mobile application used by supervisors. Notifications are sent with a 'targets' field to specify whether they should appear on desktop, mobile, or both.

Stop the Belt:

The user can stop the conveyor belt immediately through the GUI. This command is sent via WebSocket and is processed by the Raspberry Pi and relayed to the Arduino-based motor controller. The message includes the source to verify authority.

Logs:

The communication module is responsible for recording critical system events and data related to the product processing workflow. These records include received system responses, operator actions, notifications, error conditions, and product-specific processing outcomes. For each product, information such as whether an error occurred, and the corresponding error rate is logged. This enables monitoring of product performance and supports quality control processes. The logging system is maintained in an extensible format and is suitable for future integration with category-specific analysis or reporting tools.

2.3. Possible Future Features

Camera Switching:

The communication layer may support commands for dynamically switching between different camera streams, allowing operators to monitor the system from multiple angles. However, camera video streaming will not be handled by WebSocket due to performance constraints. Instead, WebSocket will only send the switching command, while the stream itself will be handled via MJPEG over HTTP.

System Health Monitoring:

The desktop client may be extended to request periodic health data (e.g., CPU temperature, memory, hardware connection status) from the Raspberry Pi and display this information in a diagnostics panel.

2.4. Technologies Used

Component	Technology
Communication Protocol	WebSocket
C++ WebSocket Library	QtWebSockets
Logging	File-based (.txt or .json logs)
Programming Language	C++17

2.5. Message Format Examples

Stop Belt Command:

```
{
  "type": "command",
  "action": "stop_belt",
  "source": "desktop",
  "target": "raspberry",
  "timestamp": "2025-04-10T12:34:56"
}
```

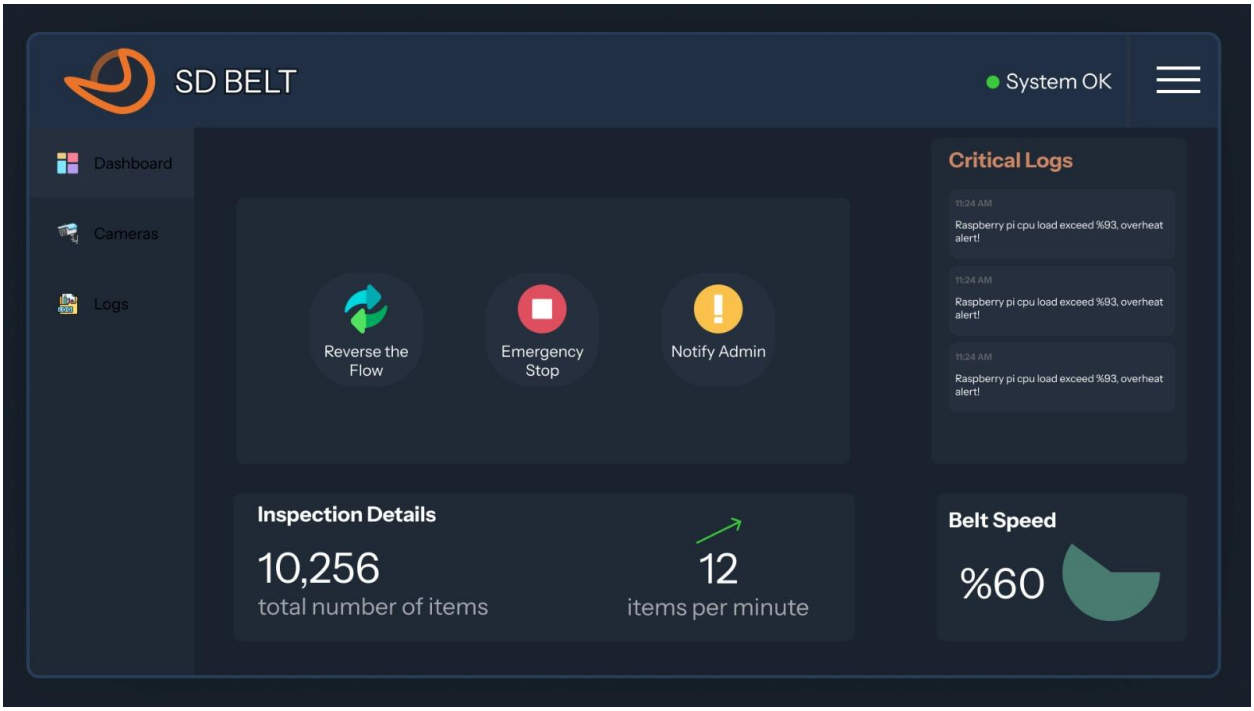
Notification Message:

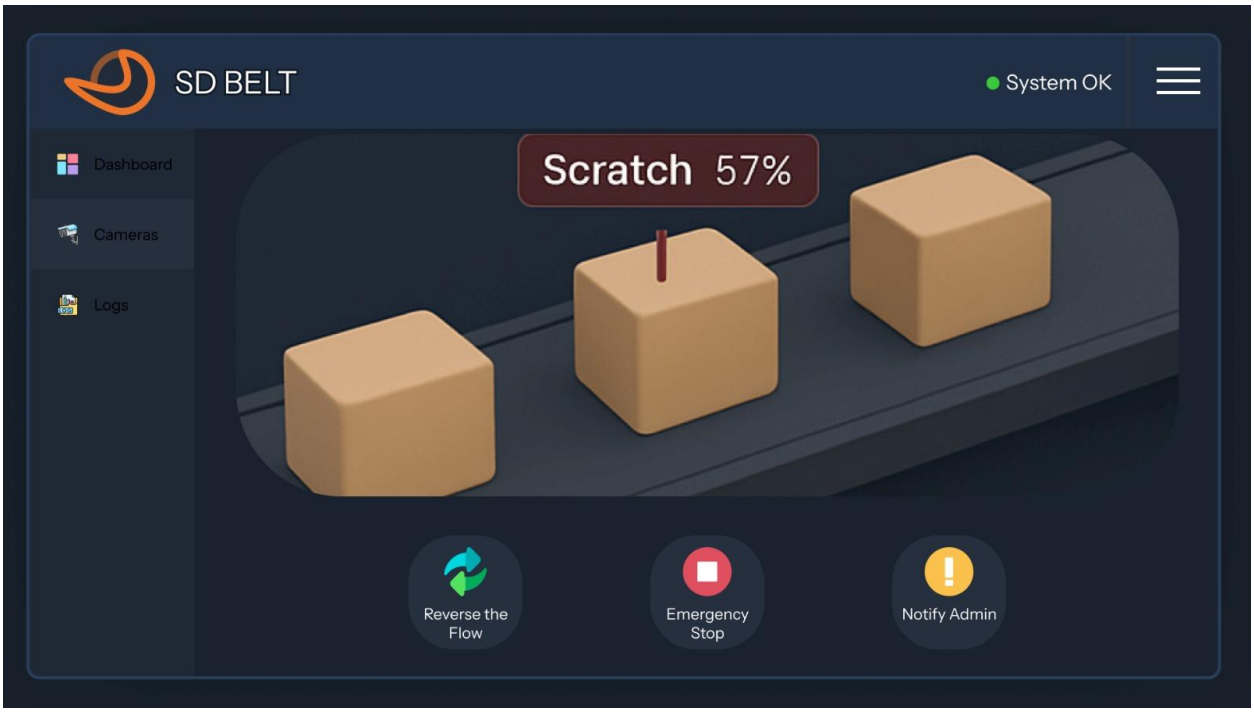
```
{
  "type": "notify",
  "message": "System overload detected",
  "severity": "critical",
  "targets": ["desktop", "mobile"],
  "timestamp": "2025-04-10T12:34:56"
}
```

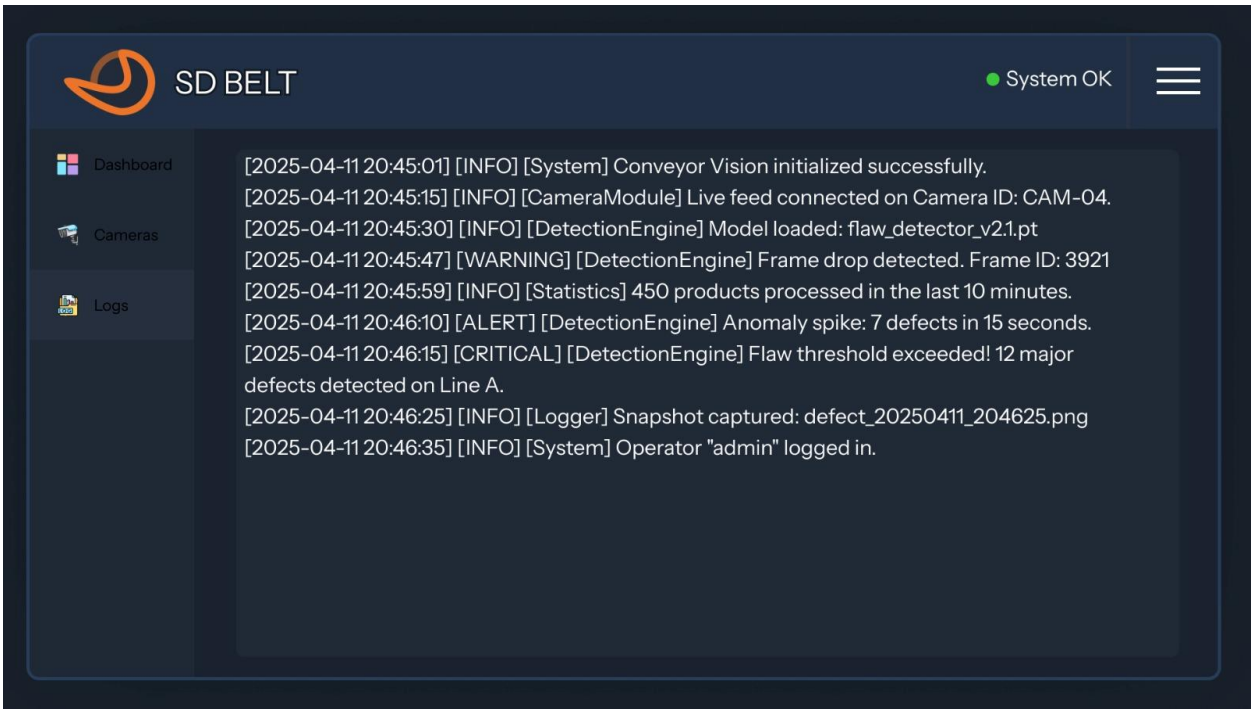
Switch Camera Command:

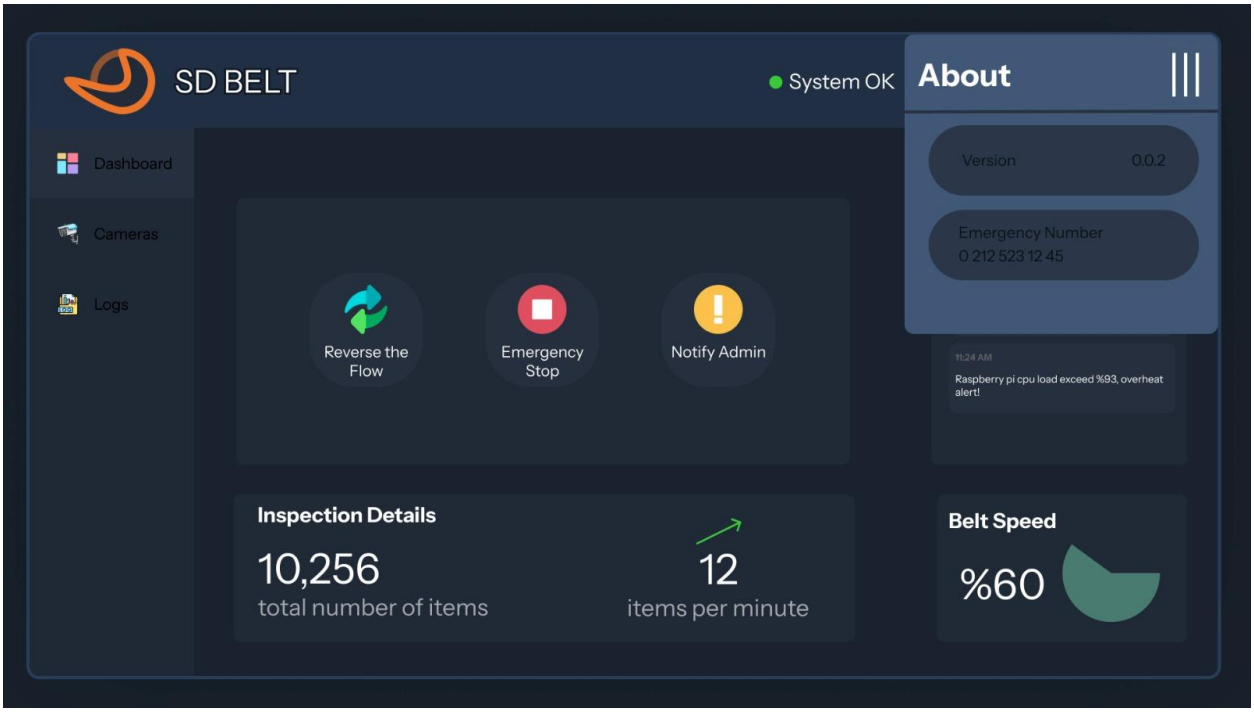
```
{
  "type": "command",
  "action": "switch_camera",
  "camera_id": "cam2"
}
```

2.6. Desktop Figma Designs






SD BELT
System OK
Dashboard
Cameras
Logs
[2025-04-11 20:45:01] [INFO] [System] Conveyor Vision initialized successfully.
[2025-04-11 20:45:15] [INFO] [CameraModule] Live feed connected on Camera ID: CAM-04.
[2025-04-11 20:45:30] [INFO] [DetectionEngine] Model loaded: flaw_detector_v2.1.pt
[2025-04-11 20:45:47] [WARNING] [DetectionEngine] Frame drop detected. Frame ID: 3921
[2025-04-11 20:45:59] [INFO] [Statistics] 450 products processed in the last 10 minutes.
[2025-04-11 20:46:10] [ALERT] [DetectionEngine] Anomaly spike: 7 defects in 15 seconds.
[2025-04-11 20:46:15] [CRITICAL] [DetectionEngine] Flaw threshold exceeded! 12 major defects detected on Line A.
[2025-04-11 20:46:25] [INFO] [Logger] Snapshot captured: defect_20250411_204625.png
[2025-04-11 20:46:35] [INFO] [System] Operator "admin" logged in.


SD BELT
System OK
About
Dashboard
Cameras
Logs
Version
0.0.2
Emergency Number
0 212 523 12 45
Reverse the Flow
Emergency Stop
Notify Admin
11:24 AM
Raspberry pi cpu load exceed %93, overheat alert!
Inspection Details
10,256
total number of items
12
items per minute
Belt Speed
%60

2.7. Test Cases for Desktop Application Module

Feature	Purpose	Input	Expected Result	Result
Notify	Verify that system notifications sent from Raspberry Pi are displayed on the desktop GUI and optionally forwarded to mobile clients.	{ "type": "notify", "message": "Test Alert", "severity": "info", "targets": ["desktop", "mobile"] }	Notification is shown on desktop GUI and logged properly.	Planned
Stop the Belt	Test whether the user can stop the SD Belt via GUI and if the command is relayed through Raspberry Pi to the motor.	{ "type": "command", "action": "stop_belt", "source": "desktop", "target": "raspberry" }	The belt stops within 2 seconds and confirmation appears in GUI or logs.	Planned
Logs	Ensure that all key actions and events are being written to the log file in a consistent format.	Trigger a stop command and receive a notification.	Both events are recorded with timestamps in the log file.	Planned
Switch Camera	Check if the system can switch active camera streams when commanded from GUI.	{ "type": "command", "action": "switch_camera", "camera_id": "cam2" }	The stream switches to cam2 and GUI reflects the new feed.	Planned
System Health Monitoring	Validate that Raspberry Pi sends health metrics and GUI	{ "type": "status", "cpu_temp": 61.2, "memory_usage":	CPU temperature and memory usage are	Planned

	displays them correctly.	45.5 }	updated in the GUI panel.	
Basic GUI Interaction with QT	Operating with interactive GUI elements such as (buttons, labels, input fields).	Verify that the user can interact with basic GUI components and see immediate visual feedback.	The GUI layout remains responsive and delivers the expected actions with visual feedbacks.	Planned
Proper Layering of QT Widgets	Making sure of UI experience is flawless and clean	Verify that layers on top of each other acts as expected based on input.	Where Pointer(Mouse) is interacted on screen will get effected by input only.	Planned

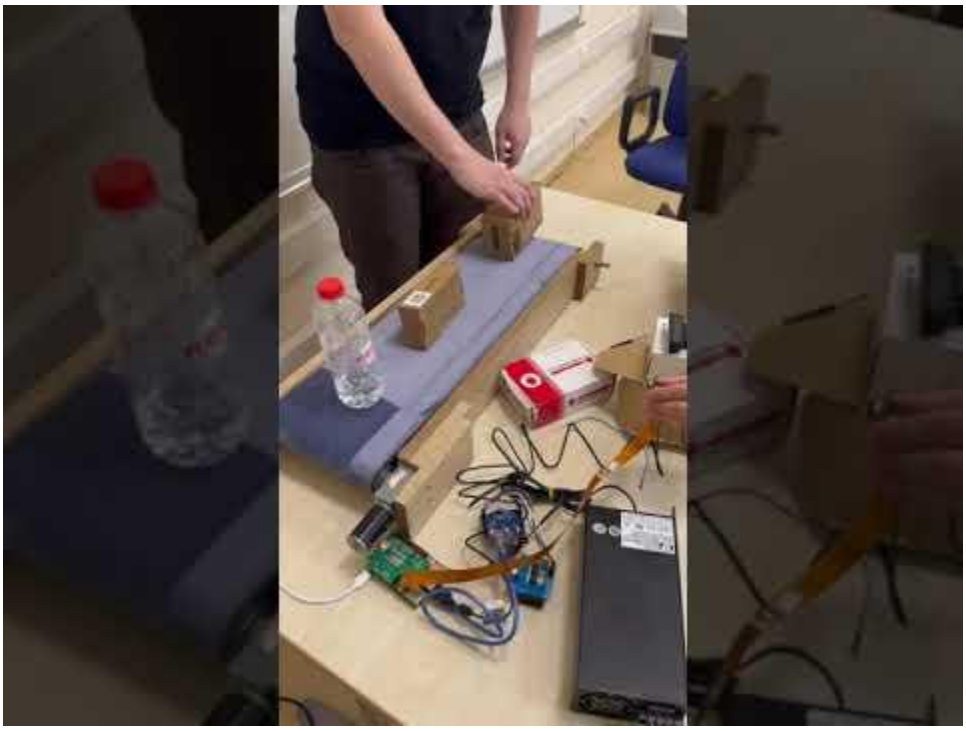
Measure WebSocket Communication Latency	WebSocket ping-pong latency check	User clicks a “Test Latency” button
		{
		"type": "ping", "timestamp": <current_time>
		}
			Server replies with a pong message
			({ "type": "pong", "timestamp": <received_time> }), around 50 ms latency expected
				Planned

3. Hardware Module

3.1. Overview

So far, we have successfully managed to operate the conveyor belt mechanism of our system. The setup includes one 24V motor, one motor driver board, and one power supply. The motor driver circuit has been connected to the Arduino, and it operates properly in response to motor control signals. Additionally, the Raspberry Pi has been integrated and physically connected to the Arduino via a serial connection. As demonstrated in the video recording, the SD Belt can be driven smoothly. In the next phase, depending on system requirements, 3 to 4 cameras will be integrated into the setup to observe products from different angles.

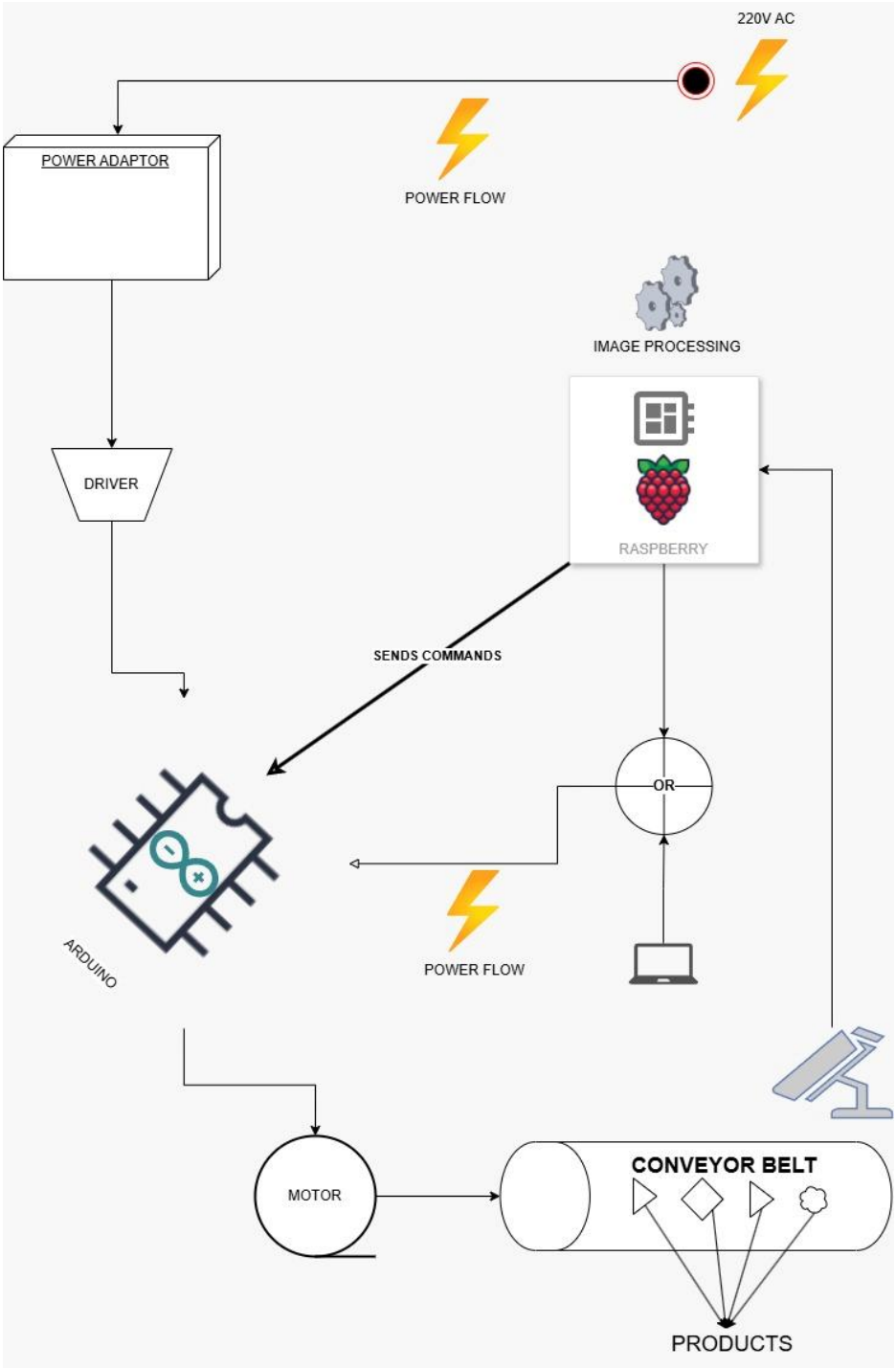
3.2. SD Belt Videos







3.3. SD Belt Workflow



3.4. Test Cases for Hardware Module

Feature	Purpose	Test Input / Action	Expected Result	Result
SD Belt Motor Activation	Verify that the motor responds correctly to control signals from Arduino.	Power on system and send "start" command from Raspberry Pi to Arduino.	SD Belt begins moving smoothly and at consistent speed.	Passed
Motor Stop Command	Ensure that the belt stops when a "stop" command is sent.	Send stop command from Raspberry Pi to Arduino while belt is in motion.	SD Belt stops within 2 seconds.	Passed
Power Supply Check	Validate that the power supply provides stable 24V under load.	Measure voltage across motor terminals while the SD Belt is running.	Voltage remains within 23–25V during operation.	Passed
Arduino-Pi Connection	Ensure serial connection between Raspberry Pi and Arduino is active.	Disconnect and reconnect the serial cable and observe command transmission.	Motor responds again after reconnection.	Passed
SD Belt Frame Stability	Confirm that mechanical frame holds the system steady.	Run the belt at full speed with a standard load placed on it.	No wobble or excessive vibration observed.	Passed
Camera Mounting Plan	Plan and verify suitable positions for multiple camera modules.	Attach mock-up cameras or placeholders at designated positions above the belt.	At least 3 camera viewpoints offer full coverage of product angles.	Planned
Camera Wiring Plan	Ensure power and data paths for camera	Plan wiring paths from Pi to 3-4 camera positions	Cabling layout is clean and Raspberry Pi ports are	Planned

	modules are feasible.	without overlap or signal interference.	reachable for each camera.	
Raspberry Pi - Arduino Serial Communication	Verify that motor control commands are reliably transmitted from Raspberry Pi to Arduino via serial interface.	Send start/stop commands over serial and observe Arduino motor response.	Motor responds accordingly to the commands without delay.	Passed
Product Transport by SD Belt	Confirm that the SD Belt can transport sample products without obstruction or slippage.	Place a sample product on the belt and activate motion.	Product is carried to the other end without misalignment or falling.	Passed
Camera-Based Product Capture	Verify that the camera records passing products during belt movement for future processing.	Activate camera while the belt is moving with sample products.	Camera captures visual data correctly during motion.	Planned
Webcam Fallback Test	Ensure that a USB webcam can be used as a backup in case of Raspberry Pi camera module failure.	Disconnect Pi camera and connect USB webcam, then run camera test application.	Video feed is displayed and functions equivalently to Pi camera module.	Planned

4. Mobile Application Module

4.1. Introduction

The SD Belt mobile application is designed to provide remote monitoring and management capabilities for production line administrators and managers. This application will enable users to monitor defect detection processes and adjust system parameters from anywhere, enhancing operational efficiency and quality control.

4.2. Technology Approach

4.2.1. Technology Stack

For the SD Belt mobile application, we have selected the following technology stack:

- **Framework**: Qt6 with QML
- **Programming Language**: C++, QML(UI)
- **Design Tools**: Figma for comprehensive UI/UX prototyping
- **Target Platforms**: Android and iOS

4.2.2. Reasons for Choosing Qt and C++

The decision to utilize Qt with C++ was made based on several factors:

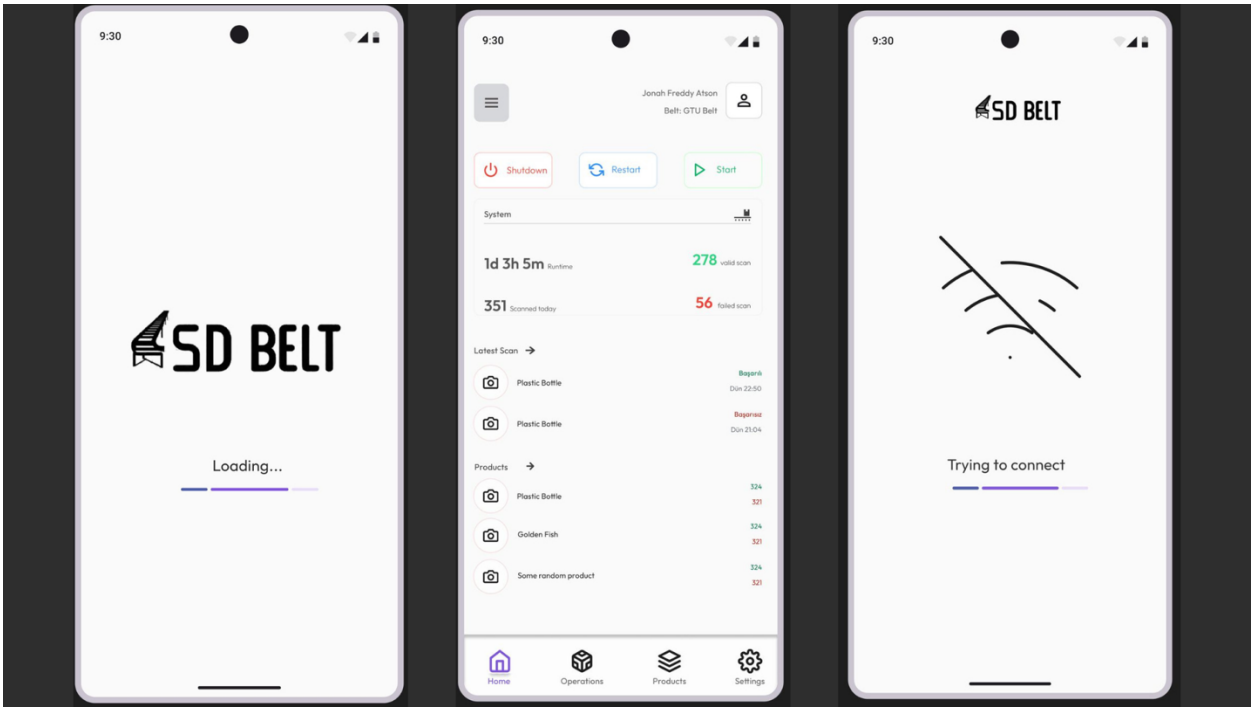
1. **Cross-platform capabilities**: Qt allows us to develop once and deploy on both Android and iOS platforms, significantly reducing development time and maintenance overhead.
2. **C++ Focused development**: Since at least 70% of the codebase is required to be in C/C++, Qt provides an ideal solution that meets our project requirements while maintaining development efficiency.

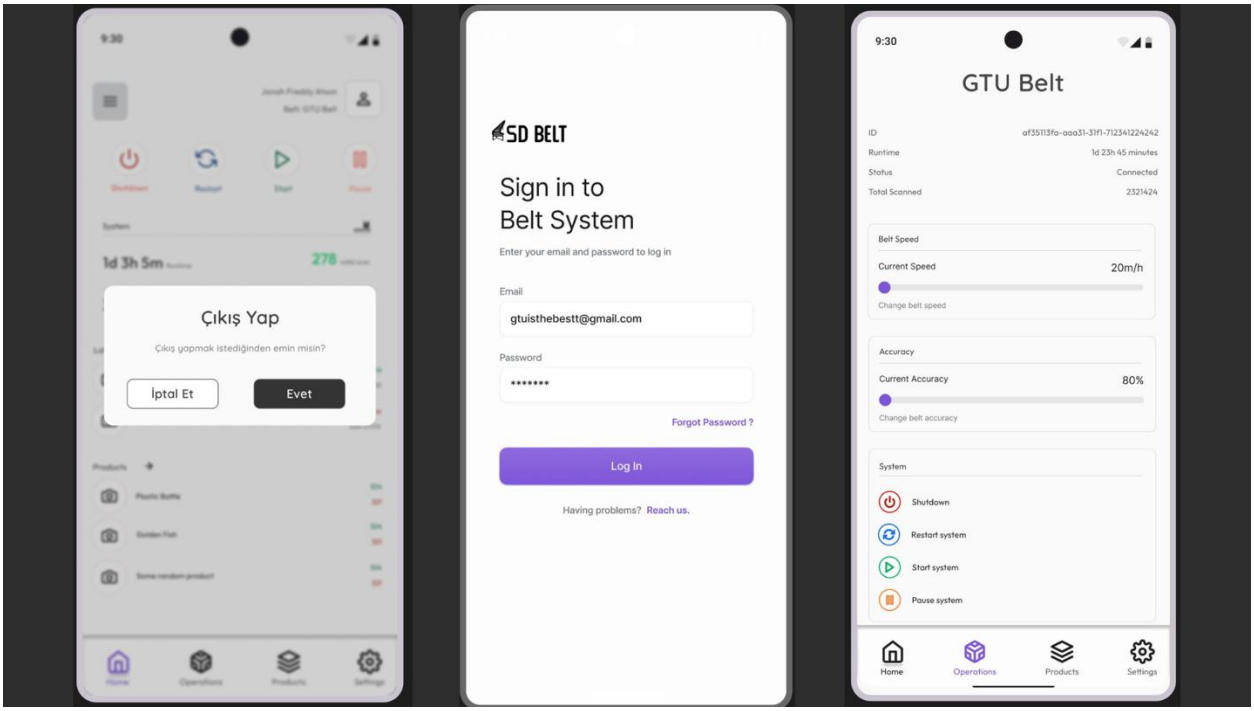
4.3. Current Progress

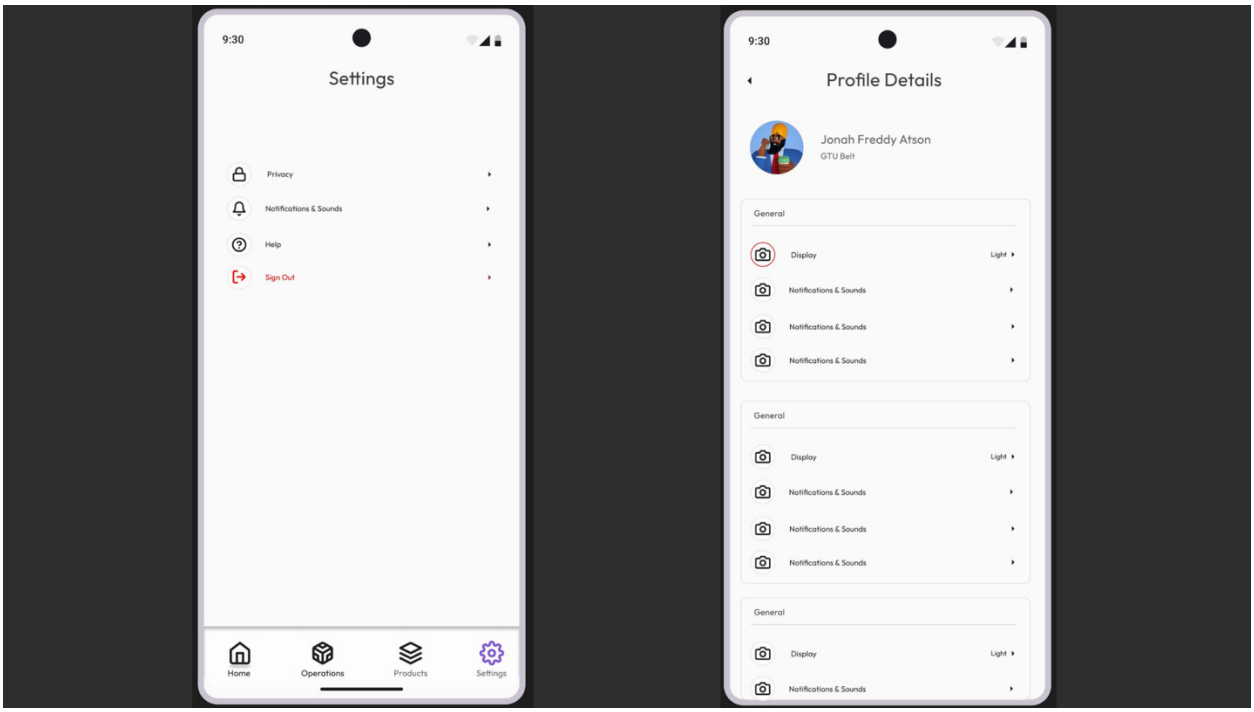
4.3.1. Design Phase

We have completed significant design work for the mobile application:

- **UI/UX Design in Figma**: Created comprehensive high-fidelity designs for almost all main application screens, including:
    - Splash screen and loading interface
    - Login/Authentication interface
    - Dashboard with real-time production metrics
    - Profile management screen
    - Settings and configuration panel







- **Design System**: Developed a consistent design language including color schemes, typography, component styles, and interactive elements to ensure visual coherence across the application.

4.3.2. Development Planning

We are currently in the planning phase for implementation:

- **Technology Research**: Completed preliminary research on Qt framework and its capabilities for mobile development.
- **Development Environment Setup**: Researching the necessary tools and configurations for Qt with C++ development for cross-platform mobile applications.
- **Architecture Planning**: Designing the software architecture that will best implement our Figma designs using Qt's capabilities.
- **Learning Process**: Since our team is not extensively familiar with Qt, we have allocated time for research and skill development to ensure a smoother development experience.

4.4. Development Roadmap

4.4.1. Phase 1: Setup and Foundation

- Configure Qt development environment for cross-platform mobile development
- Set up version control and project structure
- Create basic application skeleton with navigation framework
- Design additional pages in Figma for remaining features (real-time camera monitoring interface, etc.)
- Implement application theme and style guidelines based on our existing Figma designs

4.4.2. Phase 2: Core UI Implementation

- Develop QML components for all designed screens
- Implement the bottom tab navigation system
- Create UI components (cards, buttons, inputs, etc.)
- Implement screen transitions and basic UI flow

4.4.3. Phase 3: Functionality and Logic

- Develop authentication and user management logic
- Implement API communication layer for backend integration
- Create data models for representing production information
- Develop real-time monitoring functionality

4.4.4. Phase 4: Testing and Refinement

- Cross-platform testing on Android and iOS devices
- Bug fixing and interface refinement

4.5. Integration with Other Modules

The mobile application integrates with other SD Belt modules through:

- **Backend API Integration**: The backend module will serve as the central bridge between the physical belt system and the mobile application, facilitating all data exchange. It will handle RESTful API calls for data retrieval, system configuration, and monitoring communication, ensuring a seamless flow of information across the system.

4.6. Test Cases for Mobile Application Module

Feature	Purpose	Test Input / Action	Expected Result	Result
User Login	Verify login with credentials	Enter valid username and password	User is redirected to dashboard; token/session is created	Planned
Invalid Login	Prevent login with invalid credentials	Enter wrong password or username	Error message shown: "Invalid credentials"	Planned
Daily Scan Statistics	Validate correct rendering of product scan statistics	Backend sends data: total: 10, valid: 8	App shows: 8 valid, 2 invalid, ratio 80% valid	Planned
Belt Accuracy Display	Ensure belt accuracy is correctly calculated and shown	Backend sends accuracy = 80%	App displays "Belt Accuracy: 80%" with correct formatting	Planned
Belt Speed Visualization	Confirm belt speed updates correctly from backend	Speed rate = 3 sent from backend	App shows "Belt Speed Rate = 3" in the stats panel	Planned
Offline Mode (No Internet)	Check app response when device is offline	Turn off internet; open app	App displays "Offline - Connect to sync latest stats" message	Planned
Real-Time Throughput	Verify pipeline sends live info	Backend constantly sends latest info	App updates display accordingly without needing manual refresh	Planned

5. Database and Backend Module

5.1. Introduction

The SD Belt backend is designed as a robust and secure central communication and data management hub that connects the mobile, desktop, and hardware modules. It facilitates efficient data transfer, system integration, and real-time interactions across the entire SD Belt project. The backend module ensures high-performance processing, reliable data management, and coherent operation across all interconnected components.

5.2. Technology Approach

5.2.1. Technology Stack The SD Belt backend will utilize the following technology stack:

- Framework: Spring Boot
- Programming Language: Java
- Database: PostgreSQL
- Communication Protocol: WebSockets, RESTful APIs
- Cloud Services: AWS or Azure for deployment, scalability, and security

5.2.2. Reasons for Choosing Spring Boot and PostgreSQL

1. Comprehensive Ecosystem: Spring Boot's extensive features, including built-in security, configuration management, and powerful integration capabilities, accelerate the development process.
2. Scalable and Maintainable: Ideal for microservice architectures, Spring Boot allows the application to scale efficiently while remaining maintainable.
3. Reliability and Performance: PostgreSQL offers strong data consistency, ACID compliance, and performance tuning capabilities necessary for complex data transactions and real-time analytics.

5.3. Backend Architecture

5.3.1. Database Design The database schema will include:

- Product Details: Product identifiers, manufacturing data, and specifications.
- Quality Metrics: Real-time quality checks, defect detection results, and evaluation data.
- User Authentication: Secure management of user data, credentials, roles, and permissions.
- Historical Analytics: Long-term storage of analytics data to track production efficiency and defect patterns.

5.3.2. API and Communication Layer

- RESTful APIs: Designed for CRUD operations and interactions with mobile, desktop, and hardware modules, including authentication, data retrieval, and hardware command interfaces.
- WebSocket Integration: Implemented for real-time streaming of camera footage and sensor data to mobile and desktop applications.
- Messaging Queue: Utilized for asynchronous processing of high-volume data exchanges, ensuring stable system performance.

5.3.3. Infrastructure Planning

- Cloud Infrastructure: Cloud-based hosting on AWS or Azure, providing robust scaling capabilities, redundancy, and security.
- Continuous Integration and Continuous Delivery (CI/CD): Automated deployment pipelines established for rapid and reliable updates.

5.4. Development Roadmap

5.4.1. Phase 1: Infrastructure and Database Setup

- Docker-based environment setup for consistent development and production environments.
- PostgreSQL deployment and initial database schema configuration.
- Implementation of automated version control and CI/CD pipelines.

5.4.2. Phase 2: Core Backend Development

- Secure authentication and authorization systems using Spring Security.
- RESTful APIs development for handling CRUD operations and data integration.
- Middleware implementation for data validation, structured logging, and comprehensive error handling.

5.4.3. Phase 3: Real-time Integration with Modules

- Implement camera data streaming using WebSocket for seamless real-time video feeds to mobile and desktop clients.
- Develop APIs to handle control commands sent from the mobile and desktop apps to the hardware module (e.g., adjusting belt speed, stopping the belt).
- Ensure efficient, real-time bidirectional communication between backend, mobile, desktop, and hardware.

5.4.4. Phase 4: Testing and Optimization

- Perform extensive integration and unit testing across all API endpoints and real-time streams.
- Optimize performance for high responsiveness, minimizing latency in data transmission and processing.
- Conduct rigorous security assessments, penetration testing, and adherence to security best practices.

5.5. Integration with Other Modules The backend coordinates interactions across all SD Belt modules:

- Hardware Module: Processes and forwards camera data to other modules and receives operational commands from desktop and mobile apps.
- Mobile Application: Receives real-time camera streams and sends control commands for the hardware.
- Desktop Application: Receives real-time camera streams, provides analytics capabilities, and sends control commands to manage hardware operations effectively.

Through a well-structured architecture, robust APIs, and optimized real-time communication channels, the backend ensures efficient, secure, and reliable integration, significantly enhancing the operational effectiveness of the overall SD Belt system.

5.6. Test Cases for Database and Backend Module

Feature	Purpose	Test Input / Action	Expected Result	Result
User Authentication	Verify login and token creation	Send valid username and password to `/api/auth/login`	API returns status 200 with a valid JWT token in response	Planned
Invalid Login Handling	Prevent login with incorrect credentials	Send wrong password to `/api/auth/login`	API returns status 401 with error message "Invalid credentials"	Planned
Product Data Insertion	Ensure product defect info is correctly stored	Send POST request with product data (timestamp, defect flag etc.)	Data is stored in database's corresponding table	Planned
Real-Time Data Broadcast	Validate WebSocket sends real-time updates	Trigger a product scan event	Subscribed clients (mobile/desktop) receive WebSocket message and video	Planned
Database Constraint Check	Validate rejection of invalid data formats	Send malformed data (e.g., missing `timestamp`)	API returns 400 Bad Request; no insertion into DB	Planned

Command Handling	Confirm backend processes control commands	Send an action for controlling the belt	Command is forwarded to hardware module and acknowledged	Planned
Invalid API Endpoint	Test server response to invalid route	Send GET to a not existing endpoint	Returns 404 Not Found with a proper error message	Planned